Audit Report

Own Protocol v2

Date: **July 13 — September 9, 2026**

Introduction

eUSD is an overcollateralized dollar stablecoin issued against tokenized real-world assets (RWAs). Holders of tokenized equities lock them as collateral in the `EUSDManager` and mint eUSD against a fresh oracle price, without selling the underlying exposure. Positions sit in a per-collateral sorted list ordered by collateral ratio; a fixed annual stability fee accrues on debt and is minted to the protocol treasury, so eUSD supply always equals outstanding debt; keepers liquidate unsafe positions at a fixed bonus; and Liquity-style redemption against the riskiest positions anchors the peg at $1. `EUSD` is the token, with rate-limited ERC-7802 bridge entry points.

Collateral is priced by `ChainlinkOracleVerifier`: Chainlink feeds are the primary source, and band-limited, EIP-712-signed in-house quotes fill in only while a feed is quiet, so tokenized stocks can still be valued when their home market is closed. A compromised signer can never move the served price further than the configured band from the last Chainlink answer.

This edition covers the eUSD stablecoin and the price oracle it depends on. It consolidates the eUSD and oracle findings of the reviews completed between July 13 and September 9, 2026: the eUSD module review (September 2 — September 9) and the oracle findings of the earlier review that introduced the Chainlink-primary verifier (July 13 — August 3). At the time of review, eUSD collateral was the protocol's eTokens (tokenized stock exposure such as eSPY); findings for components since retired from the codebase are omitted.

The review combined manual analysis of every contract in scope with independent multi-agent adversarial audit passes — module-scoped, then branch-wide, then protocol-wide — with every finding adversarially validated against source. The module's core accounting was attacked directly and held: eUSD supply conservation and sorted-list integrity survived every path. Every High and Medium fix carries a regression test verified to fail without it.

Scope

Repository: `own-protocol/own-v2`

Audited Commit (eUSD module): `c1cb8e7747230cd1413081129a4caebfb948f3fc`

Audited Commit (oracle): `7d5a652a17bc6f48d0befe672120797c66362cc2`

Files:

- `src/core/EUSDManager.sol`
- `src/tokens/EUSD.sol`
- `src/core/ChainlinkOracleVerifier.sol`

File paths are as of the audited commits. Statuses are as recorded at the close of the September review; later changes to these findings are tracked in the November 2026 report.

Findings

Each issue has an assigned severity:

Critical/High issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.

Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

Low/Info issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked `RESOLVED` are fixed in the codebase and carry a regression test that fails without the fix. Issues marked `ACKNOWLEDGED` are accepted by design, with the rationale and any operational mitigations documented.

Issues Found

Critical	High	Medium	Low	Info
0	2	4	13	13

Issues Not Fixed and Not Acknowledged

Critical	High	Medium	Low	Info
0	0	0	0	0

Summary of Findings

ID	Severity	Title	Status
H-01	High	Partial redemption of an underwater position stranded unbacked debt at the list head	Fixed
H-02	High	A routine stock split silently mis-valued every eUSD position by the split ratio	Fixed
M-01	Medium	Redemption could not skip an underwater head, stalling the peg anchor	Fixed
M-02	Medium	Full-debt-only liquidation could be starved of eUSD liquidity	Fixed
M-03	Medium	Halted collateral was valued at the live feed, enabling unbacked minting	Fixed
M-04	Medium	Partial liquidation below 1 + bonus seized the full bonus, manufacturing unbacked eUSD	Fixed

L-01	**Low**	`mintPriceMaxAge` was a silent no-op inside the oracle's `clFreshWindow`	Fixed
L-02	**Low**	Sorted-list ordering drifts under lazy stability-fee accrual	Acknowledged
L-03	**Low**	No `minDebt` floor on the redemption path	Acknowledged
L-04	**Low**	Oracle `maxAnchorAge` width vs liquidation bonus (stale-anchor exits)	Acknowledged
L-05	**Low**	`setRiskParams` threshold raise assumes ADMIN sits behind the timelock	Acknowledged
L-06	**Low**	ADMIN/OPERATOR role namespace is protocol-global, not per-contract	Acknowledged
L-07	**Low**	`MINTER_ROLE` exclusivity not structurally enforced on `EUSD`	Fixed
L-08	**Low**	eToken collateral dividends were stranded in the manager with no claim path	Fixed
L-09	**Low**	Disabled collateral blocked defensive top-ups while liquidation stayed live	Fixed
L-10	**Low**	`withdrawCollateral` with debt escaped the `mintPaused` lever	Fixed
L-11	**Low**	A reverting aggregator bricks both price legs, including a fresh in-house price	Acknowledged
L-12	**Low**	Cached `clDecimals` can rot on an aggregator upgrade	Acknowledged
L-13	**Low**	A feed that dies mid-session reads as current for up to `clFreshWindow`	Acknowledged

I-01	Info	`_freshPrice` tolerates future-dated timestamps	Acknowledged
I-02	Info	Fee rounds to zero but `feeIndexSnapshot` still advances	Acknowledged
I-03	Info	Zero-fee redemption (no Liquity-style base rate)	Acknowledged
I-04	Info	`debt * (BPS + bonus)` computed outside `mulDiv`'s 512-bit space	Acknowledged
I-05	Info	`EUSD.crosschainBurn` burns from an arbitrary `from` (trusted-bridge model)	Acknowledged
I-06	Info	Bridge burn+mint pairing evades the `netBridgedIn` global cap	Acknowledged
I-07	Info	Zero-amount `crosschainMint/Burn` let any EOA emit spoofed bridge events	Fixed
I-08	Info	`setBridgeLimits` reset remaining capacity to max — instant window refill	Fixed
I-09	Info	Stability-fee accrual bypasses `debtCeiling` (Maker-style, intended)	Acknowledged
I-10	Info	`_verifyInhouseProof` lacks the per-asset staleness bound of `updatePrice`	Acknowledged
I-11	Info	`setChainlinkConfig` keeps the old in-house price cache	Acknowledged
I-12	Info	Enabling a zero side of a live bridge starts it empty (fail-safe)	Acknowledged
I-13	Info	Compromised-signer damage cap equals `bandBps` during any quiet stretch	Acknowledged

Detailed write-ups follow for every fixed finding and for the acknowledged findings whose trust boundaries are load-bearing.

Issue H-01: `_redeemFrom` stranded unbacked zero-collateral debt at the list head RESOLVED

Severity: High

Vulnerability Detail

`_redeemFrom` capped `seized` at `p.collateral` but subtracted the full `repaid` from `p.debt`, so partially redeeming an underwater position left `debt > 0, collateral == 0`. The surviving node re-entered the sorted list with key `collateral/debt = 0` — the minimum — pinning it permanently at `listHead`. Every subsequent `redeem` had to consume the head first: `seized` capped at zero, so redeemers burned eUSD for no collateral, and any redeemer with a non-zero `minCollateralOut` reverted `SlippageExceeded` and could not redeem at all. `liquidate` could not clear it (burning full debt for zero collateral is a pure loss no keeper takes) and the owner had no incentive to `closePosition`. The state is reachable by any redemption of amount in `[collateral value, debt)` against an underwater position, and an attacker could manufacture it deliberately at fair-value cost, with the toll landing on everyone after them. The root cause was an asymmetry: `liquidate` handles the same collateral cap by clearing the *full* debt, `_redeemFrom` did not.

Impact

A permanent toll — or a hard revert under slippage protection — on the peg-defense redemption path, plus unbacked eUSD accumulating in `totalDebt`.

Resolution

`_redeemFrom` now caps `repaid` to the seized collateral's value at the redemption price whenever the collateral cap fires, so a redeemer is never charged for collateral they do not receive. The unbacked residual stays on the owner's books, off-list, still counted in `totalDebt` (the supply invariant is untouched), and clearable by repay, close, liquidation, or top-up. List membership is now link-derived (`_isListed`) instead of inferred from `debt > 0`; `_reindex` inserts only when `debt > 0 && collateral > 0`; and the `_insertNode` hint check ignores stale hints pointing at off-list residuals, which previously could link a new node behind a detached predecessor with a caller-supplied hint. The redemption walk continues past a

drained head in the same call. Five regression tests cover the zombie path, fair-value walk continuation, and residual re-listing, and the list invariant now asserts no listed node has zero collateral.

Issue H-02: A routine stock split silently mis-valued every eUSD position by the split ratio RESOLVED

Severity: High

Vulnerability Detail

`EUSDManager` custodied collateral as a fixed token-address balance but valued it by ticker: every ratio and seizure computation multiplied the raw stored unit count by the per-active-unit ticker price, applying no legacy-ratio factor, and the module had no split/migration hook. When the admin performs a supported corporate action — `AssetRegistry.migrateToken(ticker, newToken, ratio)` — the deposited token becomes legacy, the external feed reports the new per-active-unit price, and every existing position keeps operating silently mispriced by `ratio`. Worked case (MCR 150%, threshold 130%, bonus 5%): a position of 2 eSPY against 800 eUSD at $600/share is worth $1,200 (CR 150%). A 2:1 forward split moves the feed to $300; the position reads 75% and any keeper liquidates it, receiving $1,200 of true value for $800 — a 50% profit, with the borrower losing the entire MCR buffer. A reverse split inverts it: positions over-valued by `ratio`, allowing withdrawal or fresh minting against phantom backing. `addCollateral` also accepted already-legacy tokens, so the mispricing was reachable without any migration, and `BorrowManager._effectivePrice` already implemented the correct scaling — the manager was the sole eToken pricer without it.

Impact

A legitimate, protocol-supported corporate action made genuinely healthy positions liquidatable at a large keeper profit (forward split) or admitted uncollateralized eUSD supply (reverse split), with the victim's `withdrawCollateral` rescue blocked by the same wrong price.

Resolution

A new private `_effectivePrice(collateral, price)` scales the per-active-unit ticker price by `legacyRatioToActive(collateral)` (ratio 0 → identity), mirroring the existing `BorrowManager` house pattern. It is applied inside the two price helpers `_freshPrice` / `_anchorPrice`, so every valuation site — withdraw, mint, liquidate,

redeem, ratio views — is covered by construction, while seizure and transfer amounts stay denominated in the token actually held. `addCollateral` now rejects already-legacy tokens (`LegacyCollateral`). The migrate-before-feed ordering, mint/deposit freeze, and legacy-disable steps are recorded as the eUSD collateral policy & split runbook. Six unit tests plus an integration suite against the real `AssetRegistry.migrateToken` + `VaultManager.applySplit` cover forward, reverse, and two-hop migrations.

Issue M-01: Redemption could not skip an underwater head, so the peg anchor stalled exactly when liquidation was unprofitable RESOLVED

Severity: Medium

Vulnerability Detail

`redeem` consumes strictly from `listHead` with no skip mechanism. Any head with CR < 100% short-changed redeemers, and a redeemer protecting themselves with an accurate `minCollateralOut` always reverted and could not reach healthy positions behind the head. This bites precisely in the crash regime where liquidating that head (CR below 100% + bonus) is also loss-making, so no rational actor cleared the blockage.

Impact

The peg-defense redemption path was inert exactly while eUSD traded below par.

Resolution

Closed by the H-01 fix without a separate change. An underwater head is now redeemed value for value (the burn is capped at its collateral's worth), it leaves the list in the same call, and the walk continues to the next position at full value — a redeemer receives exactly one dollar of collateral per eUSD across the walk and an accurate `minCollateralOut` holds. Pinned by a regression that redeems through an underwater head into a healthy position with an exact non-zero `minCollateralOut`. The unbacked residual left on the drained owner's books is tracked under H-01 as a backstop/accounting decision, not a liveness issue.

Issue M-02: Full-debt-only liquidation could be starved of eUSD liquidity RESOLVED

Severity: Medium

Vulnerability Detail

`liquidate` burned the position's entire debt from the caller in one call, with no partial option. A large borrower could therefore make themselves structurally unliquidatable: mint near the `debtCeiling` and hoard or bridge out the minted eUSD. When the position turned unsafe, no other actor could assemble the full debt — circulating supply outside the whale was smaller than the debt, minting liquidation liquidity was blocked by the ceiling the whale filled (`DebtCeilingExceeded`), redemption could only shave with whatever circulated, and raising the ceiling is a delayed ADMIN action spanning the crash. Worked case: ceiling 1M, whale mints 800k against $1.2M collateral and hoards; a 40% drop leaves $720k backing 800k debt and no path to burn 800k in one call.

Impact

The largest positions were unliquidatable in exactly the state liquidation exists for.

Resolution

Partial liquidation: `liquidate(collateral, owner, amount, hint)` repays `min(amount, debt)` (max = full close) with the same bonus math pro-rata, capped at the collateral. A partial that would leave `0 < remaining < minDebt` reverts `BelowMinimumDebt`, the remainder is re-sorted with the caller's hint, and surplus collateral refunds only on a full close. Tests include a 10,000-debt whale cleared in chunks by a keeper who never holds more than 1,000 eUSD, and the invariant handler now liquidates partially whenever a keeper cannot fund the full debt. (The partial-seizure bonus math this introduced was subsequently hardened under M-04.)

Issue M-03: Halted collateral was valued at the live feed, enabling unbacked minting above the halt price RESOLVED

Severity: Medium

Vulnerability Detail

`EUSDManager` contained no reference to VaultManager halt or pause state. When an asset is halted, its eToken's only realizable value is the frozen `assetHaltPrice` (`OwnMarket.redeemHalted`), but the Chainlink feed keeps printing the live stock price. If the live price ran above MCR × halt price, an attacker could buy halted eTokens near halt value, deposit them, mint at the live valuation, and abandon the position — the eUSD unbacked, and no rational liquidator clearing it since the seized eTokens redeem only at the halt price. Worked case: halt $100, live rallies to $160, MCR 150% → 1,000 eTokens bought for ~$100k mint 106,666 eUSD, +$6,666 per 1,000 tokens, scaling with the rally. A second wing surfaced in re-review: once a halted asset's feeds died past `maxAnchorAge`, every exit for that collateral bricked while debt remained outstanding.

Impact

Unbacked eUSD minted against halted collateral; more softly, every existing position in a halted asset over-valued whenever the live print exceeded the halt price.

Resolution

Both wings are closed in the two price helpers. `_freshPrice` (mint, withdraw-with-debt) reverts `CollateralHalted` when `VaultManager.isAssetHalted` is set. `_anchorPrice` (liquidate, redeem, ratio views) returns the legacy-ratio-scaled `assetHaltPrice` for a halted asset without consulting the oracle — the halt price is the eToken's only redeemable value, matching `BorrowManager.settleHaltedPosition`, and skipping the feed keeps exits alive after it dies. Deposits stay allowed for defensive top-ups, and repay / close / liquidate / redeem wind positions down with holders routed to `redeemHalted`. A same-day hardening also gates mint and withdraw-with-debt on `isTradingPaused`, with exits never pause-gated. Six unit tests plus an integration test against the real

`VaultManager.haltAsset`, including the worked attack case and a dead-feed exit case settling exactly at the halt price.

Issue M-04: Partial liquidation below 1+bonus seized the full bonus, manufacturing unbacked eUSD RESOLVED

Severity: Medium

Vulnerability Detail

`_seizure` computed `seized = repaid × (1 + bonus) / price` and capped it at the position's whole collateral; on a partial, the remainder stayed on the books with no refund. The liquidator chooses the amount, so on any solvent position with ratio in [100%, 100% + bonus) a partial took the full bonus out of collateral the remaining debt needed. PoC (MCR 150% / threshold 130% / bonus 5%): at ratio 101%, a full close pays the keeper +$10; a 900-eUSD partial pays +$45 and collapses the remainder to 65%, after which a follow-on redemption leaves 35 eUSD of permanently unbacked, off-list supply. On an underwater position the same sizing *enlarged* bad debt by the bonus (a 95% position: full close −$50, partial +$45 with $95 stranded). The partial strictly dominated the full close across the whole strip, there was no `owner != msg.sender` check (so debtors could self-liquidate the same way), and overnight gaps into the band are routine for single-stock eTokens with the anchor frozen off-hours.

Impact

Solvent-band liquidations manufactured unbacked eUSD and over-penalized borrowers; underwater partials increased bad debt versus the full close the shortfall-absorption rule assumed.

Resolution

`_seizure` now takes the position's debt and, on a partial, caps the seizure at the pro-rata share `coll × repaid / debt`. The cap is inert while the ratio is at or above 1 + bonus — the bonus seizure is already below pro-rata there, so M-02's chunked-liquidation liveness is unchanged — and below it the liquidator's bonus degrades to the position's cushion, so the remainder's ratio can never fall below the pre-liquidation ratio and a partial can no longer strand debt a full close would have covered. The interface's shortfall-absorption rule now holds for partials too. Both

regression tests (in-band pro-rata cap, underwater no-extra-bad-debt) fail without the cap; the existing 120% partial test pins the inert case.

Issue L-01: `mintPriceMaxAge` was silently a no-op inside the oracle's `clFreshWindow` RESOLVED

Severity: Low

Vulnerability Detail

`ChainlinkOracleVerifier.getPrice` reports `block.timestamp` as the price timestamp whenever the feed answer is younger than `clFreshWindow` (4h in the documented config), so `_freshPrice`'s age bound could never fire in that window: a 5-minute `mintPriceMaxAge` actually admitted prices up to `clFreshWindow` old. Exposure was bounded by the feed's 0.5% deviation trigger while live and the 150%/130% closed-market buffer — but the admin-facing knob did not mean what it appeared to mean.

Impact

A misleading freshness parameter, protocol-wide (the PSM and BorrowManager consume the same reads); no direct exploit.

Resolution

Semantics confirmed as intended: an answer inside `clFreshWindow` is deviation-bounded while the feed is live, so reporting it as current is sound, and bounding on the raw timestamp would push mints onto the in-house leg during normal hours. Documented on `RiskParams.mintPriceMaxAge` and `setMintPriceMaxAge`: the effective bound on the Chainlink leg is `max(mintPriceMaxAge, clFreshWindow)`; the knob bites on the in-house leg and on older Chainlink answers. No code change.

Issue L-07: `MINTER_ROLE` exclusivity was not structurally enforced on EUSD RESOLVED

Severity: Low

Vulnerability Detail

The `totalSupply == totalDebt` invariant assumes `EUSDManager` is the sole holder of `MINTER_ROLE` on the token, but plain `AccessControl` (no enumeration) cannot structurally enforce exclusivity, and nothing asserted it at deployment.

Impact

A second minter — granted by mistake or by a compromised admin — would silently break the supply-conservation invariant every other guarantee builds on.

Resolution

`DeployEusdRobinhood.s.sol` now asserts, on the freshly deployed token, that the manager holds `MINTER_ROLE`, that neither the deployer nor the token admin does, and that the deployer no longer holds `DEFAULT_ADMIN_ROLE`. Monitoring on `RoleGranted(MINTER_ROLE)` remains an ops item.

Issue L-08: eToken collateral dividends accrued to the manager with no claim path RESOLVED

Severity: Low

Vulnerability Detail

Collateral is protocol eTokens, which are dividend-bearing (`claimableRewards` / `claimRewards` / `rewardToken`). While an eToken sat as CDP collateral, its dividends accrued to the `EUSDManager` address — the current holder — but the manager had no claim, forward, or admin sweep path, so that yield was stranded for the life of every loan. This was asymmetric against the analogous `OwnMarket` escrow path, which does implement `sweepDividends`, and live-relevant: eSPY on Robinhood Chain has a `rewardToken` set.

Impact

A permanent value leak — the dividend stream on custodied collateral was unrecoverable.

Resolution

The reward model was first confirmed claim-based (rewards-per-share accumulator, non-rebasing — so `totalCollateral` cannot desync from the real balance). A new permissionless `sweepCollateralRewards(collateral)` claims the manager's accrued rewards and forwards them to the protocol treasury — the same destination rule as `OwnMarket.sweepDividends` and the borrow manager's collateral-dividend sweep — reverting `NoRewardsToSweep` when nothing is claimable. Tests cover the treasury forward against a real `EToken` dividend and the empty-sweep revert.

Issue L-09: Disabled collateral blocked defensive top-ups while liquidation stayed live RESOLVED

Severity: Low

Vulnerability Detail

`deposit` applied the `cfg.enabled` gate unconditionally, but `liquidate` has no enabled gate. After a legitimate de-listing `setCollateralEnabled(c, false)`, an indebted borrower could not top up collateral — their only risk-decreasing lever other than sourcing eUSD to repay — while keepers could still liquidate at the bonus.

Impact

In a drawdown during a de-listing window, a borrower holding spare eTokens was forced into an avoidable liquidation and paid the liquidation bonus to the keeper.

Resolution

`deposit` now reverts `CollateralDisabled` only when the caller has no debt on that collateral, so an existing debtor may always top up; `mint` keeps its unconditional gate, so a disabled collateral admits no new debt from anyone. The regression test walks a liquidatable debtor topping up to safety, the same debtor blocked from minting, and a debt-free holder blocked from depositing.

Issue L-10: `withdrawCollateral` with debt outstanding escaped the emergency mint-pause lever RESOLVED

Severity: Low

Vulnerability Detail

For an indebted position, withdrawing collateral is risk-increasing with exactly a mint's shape — the same `_freshPrice` and MCR gate — yet it checked neither `mintPaused` nor `cfg.enabled`. During a bad-oracle-price incident (a leaked signer or erroneous print, the threat the protocol's settle bands exist for), ops could throw both levers and debtors could still extract collateral against the inflated price down to MCR, leaving undercollateralized debt behind, while the identical extraction via `mint` was blocked. Worked case: a price pushed 2× lets a 100-token / $50-debt position withdraw 62.5 tokens, leaving $37.50 of real backing on $50 of debt.

Impact

The incident lever did not actually stop value extraction.

Resolution

`withdrawCollateral` with `p.debt > 0` now reverts `MintingPaused` when the mint pause is set — the same operator lever, same scope as `mint`. It is deliberately not gated on `enabled`: per the L-09 resolution, that switch means "no new exposure" and must keep letting debtors reduce risk. Pure exits (repay, close, debt-free withdrawal) stay ungated; together with the halt and trading-pause gates in `_freshPrice`, every risk-increasing action on the manager now answers to the same switches.

Issue L-11: A reverting aggregator bricks both price legs, including a fresh in-house price ACKNOWLEDGED

Severity: Low

Vulnerability Detail

`_chainlink()` lets a revert from `latestRoundData()` bubble up. A stale feed correctly fails over to the in-house leg, but a reverting one — an unset or bricked proxy — denies service to `getPrice` and blocks `updatePrice`, taking the in-house leg down with it even when a fresh signed price is available.

Impact

Availability only; funds are safe and the state is recoverable by an admin.

Resolution

Acknowledged and won't fix. A `try`/`catch` fallback was considered and declined: with no anchor the in-house leg is unusable by design anyway, so failing closed is the intended posture. Recovery is via `setChainlinkConfig` or `disableAsset`. Note that the related `ZeroMultiplier` hard-revert in the `multiplierToken` branch is a different code path and is not covered by this acceptance; it is carried separately as a lead.

Issue L-12: Cached `clDecimals` can rot on an aggregator upgrade ACKNOWLEDGED

Severity: Low

Vulnerability Detail

`setChainlinkConfig` caches the aggregator's `decimals()` once at configuration time. An aggregator upgrade behind the proxy that changed the decimal count would silently mis-scale every price for that asset by a factor of 10^n, with no on-chain signal that anything had changed.

Impact

Silent and severe mis-scaling of an asset's price if it ever occurs.

Resolution

Acknowledged — probability is very low, since all current Robinhood feeds are 8-decimal, while re-reading decimals on every price read would add cost to the hot path. Mitigated operationally: aggregator-upgrade monitoring across all feed proxies is a standing item on the ops checklist, and any reconfiguration through `setChainlinkConfig` re-caches the value.

Issue L-13: A feed that dies mid-session reads as current for up to `clFreshWindow` ACKNOWLEDGED

Severity: Low

Vulnerability Detail

The timestamp clamp assumes that the absence of an update implies the price is still within the feed's deviation band, which fails if feed infrastructure dies silently. A feed that has been quiet for fifteen minutes is indistinguishable on-chain from a dead one, so the assumption cannot be tested from within the contract.

Impact

A dead feed's last price is served as current for up to `clFreshWindow`.

Resolution

Acknowledged as inherent to the freshness semantics. `clFreshWindow` was reduced from 24.5 hours to 12 hours and then to 4 hours precisely to shrink this window, which is the only lever available in-contract. Mitigated operationally by feed-age alerting below the 4-hour bound on the ops checklist.

Issue I-05: `EUSD.crosschainBurn` burns from an arbitrary

ACKNOWLEDGED

Severity: Info

Vulnerability Detail

The ERC-7802 `crosschainBurn(from, amount)` entry point is allowance-free and consent-free: authorization is solely the caller's per-bridge burn rate limit, so any admin-authorized bridge can destroy any holder's eUSD up to its per-window `burnMaxLimit`. The NatSpec claim that a bridge "only burns from the user who asked it to bridge" is an off-chain trust assumption, not an on-chain guarantee.

Impact

A compromised bridge's per-window limit is a theft budget against arbitrary holders — the standard xERC20 / ERC-7802 trusted-bridge trade-off.

Resolution

Accepted as-is: bridge authorization is admin-gated (`setBridgeLimits`), rate-limited, and launch-disabled (no bridge limits set, `maxNetBridgedIn = 0`). Recorded so the trust assumption is explicit, the NatSpec wording is corrected, and the model is re-reviewed whenever a transport is actually authorized. The plain `EUSD.burn` is `msg.sender`-only and carries no confused-deputy path.

Issue I-06: Bridge burn+mint pairing evades the `netBridgedIn` global cap ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

A compromised bridge can burn X from a victim (allowance-free, decrementing `netBridgedIn`) and mint X to itself in the same window: net stays flat, the global cap never fires, and total supply is unchanged — invisible to supply monitoring. Widened on re-review: `netBridgedIn` is net across all bridges with no floor, so organic outflows through any honest bridge permanently grant every other bridge cumulative mint headroom — a compromised bridge's total unbacked mint is bounded by total historical outflow plus the cap, not by the cap alone.

Impact

The global ceiling bounds net inflation, not cumulative theft; per-window `min(mintMaxLimit, burnMaxLimit)` is the real theft budget.

Resolution

Accepted within the trusted-bridge model (I-05) — recorded so limit sizing and monitoring treat paired burn+mint flows as a theft budget rather than relying on the net accumulator. Fix direction if bridging is ever armed: track net per bridge (capping each bridge's cumulative net) instead of one global signed accumulator.

Issue I-07: Zero-amount `crosschainMint`/`crosschainBurn` let any EOA emit spoofed bridge events RESOLVED

Severity: Info

Vulnerability Detail

`_consumeLimit(_, 0)` passed even with a zeroed bridge config (`0 > 0` is false), so any EOA could call the ERC-7802 entry points with a zero amount and emit genuine `CrosschainMint`/`CrosschainBurn` events naming itself as the "bridge" — polluting exactly the event surface bridge monitoring and indexers watch.

Impact

No fund impact; noise injected into the bridge-monitoring event surface.

Resolution

Both ERC-7802 entry points now revert `ZeroAmount`. Test: `test_crosschain_zeroAmount_reverts`.

Issue I-08: `setBridgeLimits` reset remaining capacity to the new maxima — an instant window refill RESOLVED

Severity: Info

Vulnerability Detail

Any limit update — including a *lowering* — reset the bridge's remaining capacity to the new maxima, handing it an instant full window on top of what it had just spent: a momentary 2× per-window throughput, at its worst exactly during an incident when limits are being tightened.

Impact

A bridge's per-window budget could momentarily double at precisely the moment ops tried to reduce it.

Resolution

A fresh authorization starts with a full window; an update to a live bridge settles accrued capacity via `_available` and clamps to the new maxima, never refilling. Three tests cover lowering mid-window (clamps, no refill), raising mid-window (no refill), and re-authorization after zero (full window).

Issue I-13: Compromised-signer damage cap equals `bandBps` during any quiet stretch ACKNOWLEDGED

Severity: Info

Vulnerability Detail

The silence gate opens routinely mid-session, not only on weekends, so a compromised in-house signer can move the served price up to `bandBps` off the Chainlink anchor whenever the feed is quieter than `clSilence`.

Impact

Under signer compromise, the served price can deviate from the anchor by up to the band during any quiet stretch of the trading day.

Resolution

By design — an explicit decision that the band, not a market-hours calendar, is the security boundary. A second belt sits behind it: the `BorrowManager` and `OwnMarket` band checks against VaultManager marks, plus an instant `removeSigner`.